

```
$ spring run DomainLab --defense=0..100
```

Spring Boot × DDD

// 領域駆動設計

0%

□□□□

25%

Value Object

50%

□□□□

75%

Aggregate

100%

Event / BC

\$ cat defense-plan.md



0% /



25% /



50% /



75% → 100% /



AggregateEventBounded Context



PART 01 — 0%



Entity

Transaction Script □□□

```
1  @Entity
2  public class Order {    // □□□□□□□□
3      private String status; // □□□□
4      private double total;  // □□□□□
5      // getter / setter □□
6  }
7
8  @Service
9  public class OrderService { // □□□□□□
10     public void pay(Long id, double amount) {
11         Order order = repo.findById(id).get();
12         if (!"PENDING".equals(order.getStatus()))
13             throw new IllegalStateException();
14         order.setStatus("PAID");
15         order.setPaidAmount(amount); // □□□□□
16         emailService.send(order);
17         repo.save(order);
18     }
19 }
```

SMELL REPORT

- × □□□□□□□□□□□□ Service
- × □□□□□□ 12 □□□□□□□□
- × setStatus("SHIPPED") □□□□□
- × PM □□□□□□□□□□ setStatus

□□□□□□ → □□□□□□□□□□

PART 02 — 25%

Value Object

□□□□□□□□□□



```
1 @Entity
2 public class Order {
3     @Enumerated(EnumType.STRING)
4     private OrderStatus status; // 订单状态
5
6     @Embedded
7     private Money total;        // 订单总额
8 }
```

```
1 public record Money(BigDecimal amount, Currency currency) {
2     public Money {
3         if (amount.signum() < 0)
4             throw new IllegalArgumentException("金额不能为负");
5     }
6 }
```

订单状态枚举 — 订单总额

PART 03 — 50%





```
1  @Entity
2  public class Order {
3      @Enumerated(EnumType.STRING)
4      private OrderStatus status;
5      @Embedded private Money total;
6
7      protected Order() {} // JPA
8
9      public void pay(Money amount) {
10         if (status != OrderStatus.PENDING)
11             throw new OrderNotPayableException(id);
12         if (!amount.equals(total))
13             throw new PaymentMismatchException();
14         this.status = OrderStatus.PAID;
15     }
16
17     // setStatus()
18 }
```

Service □□

```
1  @Service
2  public class OrderService {
3      @Transactional
4      public void pay(Long id, Money amount) {
5          Order order = findOrder(id);
6          order.pay(amount);
7          //
8      }
9  }
```

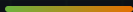
Ubiquitous Language

PM order.pay() —

→

A diagram of a 16-bit register. The top row contains 6 green squares, and the bottom row contains 10 blue squares, for a total of 16 bits.

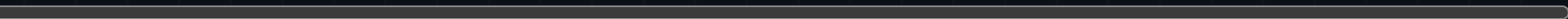
[illegible]



\$ spring run DomainLab



0% 0000



0%

25%

50%

75%

100%

x 000 double00000000

x 000000 N 0 Service0000

x setter 0000000000

x 000000000000000000

x 0000000000000000

PART 04 — 75%

Aggregate Repository

□□□□□□□□□□



```
1  @Entity
2  public class Order { // Aggregate Root
3      @OneToMany(cascade = ALL, orphanRemoval = true)
4      private List<OrderItem> items = new ArrayList<>();
5
6      public void addItem(Product product, int qty) {
7          if (status != OrderStatus.PENDING)
8              throw new OrderLockedException(id);
9          items.add(OrderItem.of(product, qty));
10         this.total = calculateTotal(); // TODO
11     }
12
13     public List<OrderItem> items() {
14         return List.copyOf(items); // TODO
15     }
16 }
```

AGGREGATE RULES

1. `addItem()` → `root` `id`
2. `addItem()` = `@Transactional` `id`
3. `addItem()` ID `id`

Repository `id`

`id` `OrderItemRepository`

PART 05 — 100%

Domain Event Bounded Context

□□□□□□□□□□

□□□□□□□□

```
1  @Entity
2  public class Order
3      extends AbstractAggregateRoot<Order> {
4
5      public void pay(Money amount) {
6          ensurePayable(amount);
7          this.status = OrderStatus.PAID;
8          registerEvent(new OrderPaid(id, total));
9      }
10 }
```

save() □□□□□

@TransactionalEventListener [commit] □□□□□

□□□□□□□□□□□□□□

```
1  @Component
2  class ShippingHandler {
3      @TransactionalEventListener
4      void on(OrderPaid event) {
5          shippingService.prepare(event.orderId());
6      }
7  }
8
9  @Component
10 class NotificationHandler {
11     @TransactionalEventListener
12     void on(OrderPaid event) {
13         emailService.sendReceipt(event.orderId());
14     } // □□□□□□□□□□
15 }
```

Bounded Context NO GRAPH □□□□□□



□□□□Product□□□□□□□□□□ — □□□□□□□□□□ 50 □□□□□□ Entity □□□□

0% vs 100%

□□	0%	100%
□□□	□□□□□□□□□□	Value Object NO DESIGN □□□□□
□□□□	setter □□□□□□	□□□□□□□□
□□□□	NO DESIGN NO DESIGN N NO DESIGN Service	□□□□□□□□□□
□□□	□□□	Aggregate □□□□□□□□
□□□□	setStatus("PAID") □□□□□	order.pay() NO DESIGN PM □□□
□□□□	□□□□□□□□	Event + Bounded Context □□



JPA Entity

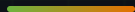
protected JPA @Embedded 100%

@Valid

@Valid DTO

100%

CRUD 25% 50%



25%

□□□□□□□□□□

50%

□□□□□□□□□□

75%

Aggregate Not Enough □□□□□

100%

Event / BC Not Enough □□□□□□□

□□□□□□□□□□□□□□□□□□

Defense complete.

```
❏❏❏❏❏❏❏ /springboot/ddd-in-spring-boot
```

```
$ spring run DomainLab --defense=100
```

```
value object → entity → aggregate → event
```